

基于精英引导的改进哈里斯鹰优化算法^{*}

李雨恒, 高尚, 孟祥宇

(江苏科技大学计算机学院, 江苏 镇江 212100)

摘要:针对哈里斯鹰优化算法(HHO)易陷入局部最优和收敛速度慢的问题,提出一种基于精英引导的改进哈里斯鹰优化算法(EHHO)。首先,引入精英反向学习,以精英中心为对称中心进行反向学习来优化种群结构,增强算法跳出局部最优的能力;其次,引入精英演化策略,以精英个体为主体进行基于高斯随机突变的演化来提升种群质量,加快算法收敛速度;最后,引入自适应机制,动态调整精英演化策略中2种演化方式的选择概率,以提升算法稳定性。为验证改进算法的有效性,选取15个基准函数进行仿真实验。实验结果表明,改进算法在寻优性能和鲁棒性上均有明显提升,在优化算法中具有一定竞争力。

关键词:哈里斯鹰优化算法;精英反向学习;精英演化策略;高斯随机突变;自适应机制

中图分类号:TP301.6

文献标志码:A

doi:10.3969/j.issn.1007-130X.2024.02.018

An improved Harris hawks optimization algorithm based on elite guidance

LI Yu-heng, GAO Shang, MENG Xiang-yu

(School of Computer, Jiangsu University of Science and Technology, Zhenjiang 212100, China)

Abstract: Aiming at the problems that Harris Hawks Optimization (HHO) is easy to fall into local optimization and has slow convergence speed, an improved Harris Hawks Optimization algorithm based on elite guidance (EHHO) is proposed. Firstly, elite opposite learning is introduced, and the elite center is used as the symmetrical center for opposite learning to optimize the population structure and enhance the ability of the algorithm to jump out of local optimum. Secondly, the elite evolution strategy is introduced, and the evolution based on Gaussian random mutation is carried out with elite individuals as the main body to improve the quality of the population and improve the convergence speed of the algorithm. Finally, an adaptive mechanism is introduced to dynamically adjust the selection probability of the two evolution modes in the elite evolution strategy to improve the stability of the algorithm. To verify the effectiveness of the improved algorithm, 15 benchmark functions are selected for simulation experiments. The experimental results show that the improved algorithm has obvious improvement in optimization performance and robustness, and has certain competitiveness in optimization algorithms.

Key words: Harris hawks optimization (HHO) algorithm; elite opposite learning; elite evolution strategy; Gaussian random mutation; adaptive mechanism

1 引言

元启发式算法具有原理简单、适用性广、无需

梯度信息和避免局部最优能力强的优点,常用于解决传统优化算法难以有效求解的多峰值、不连续和不可微的工程问题^[1,2]。群智能算法是一类基于种群的元启发式算法,通过模拟生活在种群中生物

^{*} 收稿日期:2022-09-23;修回日期:2023-02-09

基金项目:国家自然科学基金(62176107)

通信作者:高尚(gaoshang@sohu.com)

通信地址:212100 江苏省镇江市江苏科技大学计算机学院

Address: School of Computer, Jiangsu University of Science and Technology, Zhenjiang 212100, Jiangsu, P. R. China

的社会行为来在问题空间内随机搜索问题的解。典型的群智能算法有模拟鸟群觅食行为的粒子群优化算法^[3],其被广泛应用于大数据背景下的关联规则挖掘^[4];有模拟座头鲸捕食行为的鲸鱼优化算法^[5],其可用于海上物流配送路径的优化^[6]。根据没有免费的午餐定理^[7],没有优化算法是通用的最优算法,所以不断有研究人员受到自然现象的启发而提出新的优化算法,这些算法为解决优化问题提供了新的思路和灵感^[8]。

哈里斯鹰优化 HHO(Harris Hawks Optimization)算法是 Heidari 等^[9]于 2019 年提出的一种新的群智能算法,算法的主要灵感来源于哈里斯鹰在捕捉猎物时的合作行为及突袭策略。在追逐猎物的过程中,几只合作的哈里斯鹰会从不同方向包围猎物,并根据环境的变化和猎物的逃跑情况选择不同策略来突袭猎物。哈里斯鹰算法具有原理简单、参数少和易实现等优点,但同时也存在收敛精度低、收敛速度慢和易陷入局部最优等缺点。为了改善上述缺点,Li 等^[10]用精英进化策略来提高算法的收敛速度和跳出局部最优的能力;郭雨鑫等^[11]用精英反向学习机制来提升算法全局寻优能力,用黄金正弦算法来增强算法局部开发能力;陈功等^[12]用互利共生思想来加快算法收敛速度,用透镜成像反向学习策略提高算法跳出局部最优的能力;汤安迪等^[13]用精英等级策略提升算法收敛速度和收敛精度,用高斯随机游走来有效跳出局部最优;朱诚等^[14]用趋化校正机制来提高算法收敛精度。上述改进策略都在一定程度上改善了原算法的缺点,但改进算法的性能和鲁棒性仍有一定提升空间。

本文针对哈里斯鹰优化算法存在的问题,提出一种基于精英引导的改进哈里斯鹰优化 EHHO(improved Harris Hawks Optimization based on Elite guidance)算法,从 3 个方面进行改进:(1)引入精英反向学习,生成反向种群与当前种群竞争来增加种群多样性,从而增强算法跳出局部最优的能力;(2)引入精英演化策略,用精英解引导种群进化来提升种群质量,从而增强算法开发能力;(3)引入自适应机制,动态调整开发阶段不同搜索策略的选择概率,从而更好地平衡算法的开发与探索。

2 哈里斯鹰优化算法

HHO 算法模拟了哈里斯鹰种群采用不同策略搜寻并捕食猎物的合作狩猎行为,包含探索阶段

和开发阶段,并根据猎物逃逸能量的变化实现 2 个阶段之间的转换。

2.1 探索阶段

探索阶段,哈里斯鹰在搜索空间中以同等概率采用 2 种策略来搜寻猎物。一种策略是根据种群其他成员和猎物的位置来更新位置;另一种策略是在种群分布范围内随机更新位置,其形式化如式(1)和式(2)所示:

$$\mathbf{X}(t+1) = \begin{cases} \mathbf{X}_r(t) - r_1 \mid \mathbf{X}_r(t) - 2r_2\mathbf{X}(t) \mid, q \geq 0.5 \\ (\mathbf{X}_p(t) - \mathbf{X}_m(t)) - r_3(\mathbf{LB} + r_4(\mathbf{UB} - \mathbf{LB})), \text{else} \end{cases} \quad (1)$$

$$\mathbf{X}_m(t) = \frac{1}{N} \sum_{i=1}^N \mathbf{X}_i(t) \quad (2)$$

其中, $\mathbf{X}(t+1)$ 和 $\mathbf{X}(t)$ 分别是第 $t+1$ 次迭代和第 t 次迭代时哈里斯鹰的位置向量, $\mathbf{X}_p(t)$ 是第 t 次迭代时猎物的位置向量, $\mathbf{X}_r(t)$ 是第 t 次迭代时哈里斯鹰种群中随机选择的一个哈里斯鹰的位置向量, $\mathbf{X}_m(t)$ 是第 t 次迭代时哈里斯鹰种群中所有哈里斯鹰的平均位置向量, r_1 、 r_2 、 r_3 、 r_4 和 q 都是(0,1)内的随机数, $\mathbf{UB}$ 和 $\mathbf{LB}$ 是搜索空间上下界。

2.2 探索阶段和开发阶段的转换

HHO 算法根据猎物逃逸能量的大小判断算法是进入探索阶段还是开发阶段,如式(3)所示:

$$E = 2E_0 \left(1 - \frac{t}{T}\right) \quad (3)$$

其中, E 是猎物的逃逸能量, E_0 是初始状态的逃逸能量, t 是当前迭代次数, T 是最大迭代次数。在 HHO 算法的每次迭代中, E_0 在(-1,1)内随机变化。当 $E \geq 1$ 时,HHO 算法进入探索阶段,哈里斯鹰对猎物进行搜寻;当 $E < 1$ 时,HHO 算法进入开发阶段,哈里斯鹰对猎物进行捕食。

2.3 开发阶段

HHO 算法使用 4 种策略来模拟哈里斯鹰捕捉猎物的过程,分别是:软围攻、硬围攻、突袭式软围攻和突袭式硬围攻。猎物总是试图逃离危险环境,用(0,1)内的随机数 r 表示猎物在哈里斯鹰发动突袭前逃脱的情况:成功逃脱($r < 0.5$)或逃脱失败($r \geq 0.5$)。

软围攻:当 $r \geq 0.5$, $|E| \geq 0.5$ 时,猎物逃逸能量充足,并试图通过随机跳跃来误导哈里斯鹰,但未能成功;哈里斯鹰通过软围攻来消耗猎物体力,为后续发动突袭做准备,如式(4)和式(5)所示:

$$\begin{aligned} \mathbf{X}(t+1) &= \Delta \mathbf{X}(t) - E |J \mathbf{X}_p(t) - \mathbf{X}(t)| \quad (4) \\ \Delta \mathbf{X}(t) &= \mathbf{X}_p(t) - \mathbf{X}(t) \quad (5) \end{aligned}$$

其中, $\Delta \mathbf{X}(t)$ 是第 t 次迭代时猎物与哈里斯鹰之间的位置向量之差, $J = 2(1 - r_5)$ 是猎物逃逸过程中随机跳跃的强度, r_5 是 $(0, 1)$ 内的随机数。

硬围攻: 当 $r \geq 0.5$, $|E| < 0.5$ 时, 猎物逃逸能量不足, 哈里斯鹰通过硬围攻逼近猎物, 为后续发动突袭做准备, 如式(6)所示:

$$\mathbf{X}(t+1) = \mathbf{X}_p(t) - E |\Delta \mathbf{X}(t)| \quad (6)$$

突袭式软围攻: 当 $r < 0.5$, $|E| \geq 0.5$ 时, 猎物成功逃脱软围攻, 且逃逸能量依旧充足, 哈里斯鹰通过突袭式软围攻来抓捕猎物, 如式(7)~式(9)所示:

$$\mathbf{Y} = \mathbf{X}_p(t) - E |J \mathbf{X}_p(t) - \mathbf{X}(t)| \quad (7)$$

$$\mathbf{Z} = \mathbf{Y} + \mathbf{S} \times LF(D) \quad (8)$$

$$\mathbf{X}(t+1) = \begin{cases} \mathbf{Y}, F(\mathbf{Y}) < F(\mathbf{X}(t)) \\ \mathbf{Z}, F(\mathbf{Z}) < F(\mathbf{X}(t)) \end{cases} \quad (9)$$

其中, D 是待求解问题的维度; $\mathbf{S}$ 是 D 维向量, 每一维都是 $(0, 1)$ 内的随机数; $LF(\cdot)$ 是莱维飞行函数; $F(\cdot)$ 是适应度函数。

突袭式硬围攻: 当 $r < 0.5$, $|E| < 0.5$ 时, 猎物成功逃脱硬围攻, 但逃逸能量已经不足, 哈里斯鹰通过突袭式硬围攻来抓捕猎物, 如式(10)~式(12)所示:

$$\mathbf{X}(t+1) = \begin{cases} \mathbf{Y}, F(\mathbf{Y}) < F(\mathbf{X}(t)) \\ \mathbf{Z}, F(\mathbf{Z}) < F(\mathbf{X}(t)) \end{cases} \quad (10)$$

$$\mathbf{Y} = \mathbf{X}_p(t) - E |J \mathbf{X}_p(t) - \mathbf{X}_m(t)| \quad (11)$$

$$\mathbf{Z} = \mathbf{Y} + \mathbf{S} \times LF(D) \quad (12)$$

3 改进的哈里斯鹰优化算法

EHHO 算法依旧通过逃逸能量实现探索阶段和开发阶段之间的转换, 不同的是 EHHO 算法在每一轮迭代开始前用精英反向学习对种群位置进行优化, 并在开发阶段用精英演化策略替换原开发策略进行种群位置更新, 以及通过自适应机制调节精英演化策略中 2 种不同演化方式的选择概率。

3.1 精英反向学习

反向学习是 Tizhoosh^[15] 于 2005 年提出的一种新的机器智能的解决方案, 通过考虑当前解的反向解来扩大搜索范围, 能以更大概率找到更优解。将反向学习策略与群智能算法结合, 能有效增加种群多样性和质量, 使得算法能够迅速搜索到最优区域。反向解的计算如式(13)所示:

$$\tilde{\mathbf{X}} = \mathbf{A} + \mathbf{B} - \mathbf{X} = \frac{\mathbf{A} + \mathbf{B}}{2} + \left(\frac{\mathbf{A} + \mathbf{B}}{2} - \mathbf{X} \right) \quad (13)$$

其中, $\mathbf{A}$ 、 $\mathbf{B}$ 分别为搜索空间的下界和上界, $\mathbf{X}$ 为当前解, 即反向解与当前解关于搜索空间中心对称。

基本反向学习以搜索空间中心为对称中心进行搜索, 虽然扩大了搜索范围, 但缺乏针对性, 搜索比较盲目, 找到更优解的概率仍有待提升。为了增加反向学习找到更优解的概率, 本文提出一种精英反向学习方式, 以精英中心为对称中心进行反向学习。

基本反向学习中的对称中心始终保持为搜索空间中心, 其不随算法迭代动态变化。而精英反向学习中的对称中心为精英中心, 其随算法迭代动态变化。精英中心的计算如式(14)所示:

$$\mathbf{X}_c = \begin{cases} \mathbf{X}_{o1}, 0 < r < 1/3 \\ \mathbf{X}_{o2}, 1/3 \leq r \leq 2/3 \\ \mathbf{X}_{o3}, 2/3 < r < 1 \end{cases} \quad (14)$$

其中, $\mathbf{X}_{o1}$ 、 $\mathbf{X}_{o2}$ 、 $\mathbf{X}_{o3}$ 为上一轮迭代时种群中的 3 个最优解(当前最优解), r 为 $(0, 1)$ 内随机数。3 个当前最优解蕴含上一轮迭代中最重要的信息, 每次反向学习时随机选择一个当前最优解作为精英中心, 以充分利用上一轮迭代的搜索成果, 增加反向学习成功的机率。

基本反向学习中当前解仅有一个对应的反向解, 如果当前种群分布集中, 那么其反向种群也会分布集中, 这种情况下通过寻找反向解来得到更优解的概率不高。精英反向学习中引入随机参数, 使得当前解对应一个反向区域, 且该区域的直径为当前解到精英中心的距离。当距离大时, 反向学习增强探索能力; 当距离小时, 反向学习增强开发能力和跳出局部最优的能力。每次反向学习从对应区域中随机生成一个解作为反向解, 当反向区域与最优区域有重合时, 能增加反向学习成功率。精英反向解的计算如式(15)所示:

$$\tilde{\mathbf{X}} = 2r_1 \mathbf{X}_c - r_2 \mathbf{X} \quad (15)$$

其中, r_1 、 r_2 均为 $(0, 1)$ 内的随机数, $\mathbf{X}_c$ 为式(14)所得精英中心。精英反向学习的一个示例如图 1 所示, 该示例为三维空间中通过精英反向学习产生的 1 000 个反向位置的分布情况。

基本反向学习中通过比较当前解及其反向解的适应度值来选择保留更优的那个解, 但计算每轮迭代中所有当前解和反向解的适应度值会增加算法的时间开销; 此外, 如果算法陷入局部最优, 在当前解和反向解中优先保留反向解更有可能帮助算

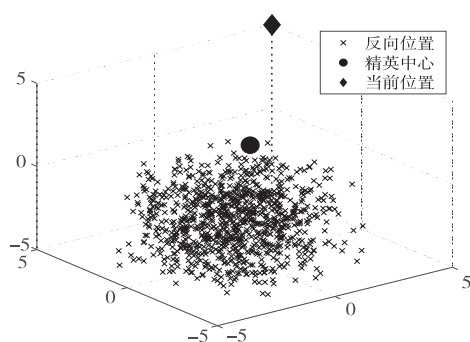


Figure 1 Elite opposite learning

图1 精英反向学习

法跳出局部最优。

为了尽量减少因反向学习而增加的计算适应度值的次数,同时充分利用反向解跳出局部最优的能力,本文设计一种反向优先策略:先计算反向解的适应度值,如果其小于保留解的历史最优的适应度值,则不再计算当前解的适应度值并直接保留反向解;否则保留当前解并计算其适应度值。保留解计算公式如式(16)所示:

$$\mathbf{X}^* = \begin{cases} \tilde{\mathbf{X}}, f(\tilde{\mathbf{X}}) < fitness \\ \mathbf{X}, f(\mathbf{X}) \geq fitness \end{cases} \quad (16)$$

其中, $f(\cdot)$ 为适应度函数, $fitness$ 为该保留解迭代过程中的最优适应度值。

3.2 精英演化策略

本文基于进化算法的核心原则,提出一种精英演化策略,以同等概率分别采用2种演化方法来更新种群,充分利用精英解引导种群演化来提升种群质量。精英演化策略保留算法迭代中的3个最优解 $\mathbf{X}_{e1}$ 、 $\mathbf{X}_{e2}$ 和 $\mathbf{X}_{e3}$ 为精英解(历史最优解),且这3个精英解的适应度值满足 $f(\mathbf{X}_{e1}) \leq f(\mathbf{X}_{e2}) \leq f(\mathbf{X}_{e3})$ 。

3.2.1 精英交叉演化

精英交叉演化的核心原则是先进进行基因交叉再进行基因突变。基因交叉从3条精英染色体获取优秀基因重组成新的染色体;而基因突变则使新的染色体进行小范围的局部变异产生精英交叉演化的染色体。因此,该方法强调开发能力,从而加快算法的收敛速度。该方法包括以下2个步骤:

基因交叉重组以50%的概率分别获取精英染色体 $\mathbf{X}_{e2}$ 和 $\mathbf{X}_{e3}$ 的基因来产生新染色体 $\mathbf{X}_{c1}$,然后以 $(1-p)$ 和 p 的概率分别获取精英染色体 $\mathbf{X}_{e1}$ 和 $\mathbf{X}_{c1}$ 的基因来重组成新染色体 $\mathbf{X}_{c2}$,计算公式如式(17)~式(20)所示:

$$ip = 1 - t/T \quad (17)$$

$$p = 0.2 \times r \times ip \quad (18)$$

$$\mathbf{X}_{c1} = (0.5 \otimes \mathbf{X}_{e2}) \oplus (0.5 \otimes \mathbf{X}_{e3}) \quad (19)$$

$$\mathbf{X}_{c2} = [(1-p) \otimes \mathbf{X}_{e1}] \oplus [p \otimes \mathbf{X}_{c1}] \quad (20)$$

其中, ip 是随迭代次数变化的参数, p 是获取不同染色体的基因产生新染色体的概率, $\oplus$ 表示对染色体进行基因交叉重组, $\otimes$ 表示进行基因交叉重组的染色体的基因以一定的概率被选择。

基因局部突变使新的染色体 $\mathbf{X}_{c2}$ 进行小范围的局部变异,以产生精英交叉演化的染色体 $\mathbf{X}_{c3}$,其变异方向与染色体 $\mathbf{X}$ 相关,变异幅度是从高斯分布 $N(0, 0.1)$ 中生成的随机数,计算公式如式(21)所示:

$$\mathbf{X}_{c3} = \mathbf{X}_{c2} + N_1 |\mathbf{X}_{c2} - \mathbf{X}| \quad (21)$$

其中, $\mathbf{X}_{c2}$ 是由式(20)得到的新染色体, N_1 是均值为0、标准差为0.1的高斯分布中的随机数。精英交叉演化的一个示例如图2所示。

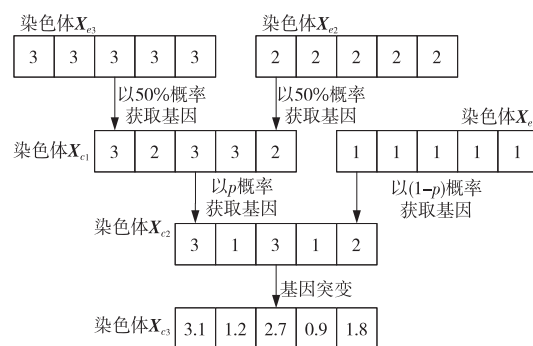


Figure 2 Elite cross evolution

图2 精英交叉演化

3.2.2 精英融合演化

精英融合演化的核心原则是先进进行基因突变再进行基因融合。基因突变使精英染色体进行较小范围的局部变异产生新的染色体;而基因融合则从精英染色体和新的染色体获取基因片段融合成新的基因,从而产生精英融合演化的染色体。因此,该方法强调探索能力,从而提高算法跳出局部最优的能力。该方法包括以下2个步骤:

基因局部突变使精英染色体 $\mathbf{X}_{e1}$ 进行较小范围的局部变异,以产生新的染色体 $\mathbf{X}_{f1}$,其变异方向与搜索空间中心 $\mathbf{CL}$ 相关,变异幅度是从高斯分布 $N(0, 1)$ 中生成的随机数,计算公式如式(22)所示:

$$\mathbf{X}_{f1} = \mathbf{CL} + N_2 |\mathbf{CL} - \mathbf{X}_{e1}| \quad (22)$$

其中, N_2 是服从均值为0、标准差为1的高斯分布中的随机数。

基因融合重组使精英染色体 $\mathbf{X}_{e1}$ 进行小幅度定向变异,使染色体 $\mathbf{X}_{f1}$ 进行大幅度定向变异,再

获取定向变异后的染色体的全部基因产生基因融合重组的染色体,计算公式如式(23)所示:

$$\mathbf{X}_{f2} = (1 + r'_1 \times ip) \times \mathbf{X}_{e1} + r'_2 \times ip \times \mathbf{X}_{f1} \quad (23)$$

其中, r'_1 和 r'_2 为 $(-1, 1)$ 内的随机数, ip 是由式(17)得到的参数, $\mathbf{X}_{f1}$ 是由式(22)得到的新染色体。精英融合演化的一个示例如图3所示。

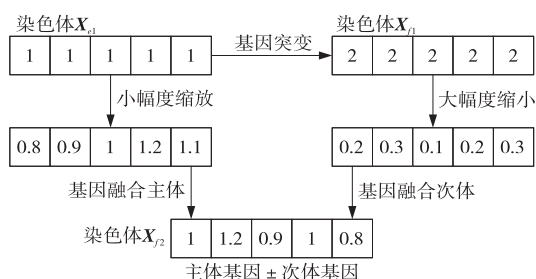


Figure 3 Elite fusion evolution

图3 精英融合演化

3.3 自适应机制

精英演化策略包含精英交叉演化和精英融合演化2种演化方式,这2种演化方式在不同情形下具备不同的寻优性能。为了充分地发挥这2种演化方式的寻优性能,本文设计一种自适应机制,通过调整选择概率 sp 来改变使用2种演化方式进行位置更新的概率。

算法开始迭代时,以同等概率采用2种演化方式更新个体位置,并在每轮迭代开始时判断上轮迭代是否更新历史最优位置,如果更新了历史最优位置,则认为算法未陷入局部最优,不调整选择概率 sp ; 否则认为算法已陷入局部最优,并试图通过调整选择概率 sp 来改变使用2种演化方式更新个体位置的概率,以寻求提升算法跳出局部最优的可能性,计算公式如式(24)所示:

$$sp = \begin{cases} sp - 0.001 \times 2^n, & diff < 0 \\ sp + 0.001 \times 2^n, & diff > 0 \end{cases} \quad (24)$$

其中,选择概率 sp 是 $[0, 1]$ 内的数,其初始值为0.5;参数 n 是连续陷入局部最优的迭代轮数;参数 $diff$ 为上一轮迭代中使用精英交叉演化和精英融合演化更新个体位置的次数之差;如果选择概率 sp 的值更新后超出 $[0, 1]$,将其修正为其最接近的边界值,同时将计数器 n 置0。当算法陷入局部最优时,算法无法直接判断哪种演化方式更适合当前情形,但可推测上一轮迭代中实际使用次数更少的演化方式更有可能帮助算法跳出局部最优,所以增大使用该方法更新个体位置的概率,且增大幅度随算法连续陷入局部最优的迭代轮数的增大而增大。

为了增强探索阶段算法的探索能力,本文对探索阶段更新公式增加随机扰动,使得探索区域更加均衡,以准确找到最优区域。改进后的计算公式如式(25)所示:

$$\mathbf{X}(t+1) = \begin{cases} r_1 \mathbf{X}_r(t) - r_2 |\mathbf{X}_r(t) - 2r_3 \mathbf{X}(t)|, & q \geq 0.5 \\ r_4 (\mathbf{X}_p(t) - \mathbf{X}_m(t)) - r_5 (\mathbf{LB} + r_6 (\mathbf{UB} - \mathbf{LB})), & \text{else} \end{cases} \quad (25)$$

其中, $r_1, r_2, r_3, r_4, r_5, r_6$ 为 $(0, 1)$ 内随机数,其余参数定义同式(1)。

3.4 EHHO 实现步骤

综合上述改进策略,本文所提出的 EHHO 优化算法伪代码如算法1所示。

算法1 EHHO 算法

Input: 种群大小 N , 最大迭代次数 T , 适应度函数 $f(\cdot)$ 及相关参数。

Output: 猎物位置 $\mathbf{X}_{e1}$ 及其适应度值 $f(\mathbf{X}_{e1})$ 。

```

1  初始化哈里斯鹰种群位置  $\mathbf{X}_i (i = 1, 2, 3, \dots, N)$ , 历史最优位置  $\mathbf{X}_{e1}, \mathbf{X}_{e2}, \mathbf{X}_{e3}$ , 当前最优位置  $\mathbf{X}_{o1}, \mathbf{X}_{o2}, \mathbf{X}_{o3}$  和选择概率  $sp$ ;
2  While ( $t < T$ ) do {
3    For (each  $\mathbf{X}_i$ ) do {
4      进行精英反向学习, 用式(15)产生反向解, 用式(16)更新位置;
5      更新历史最优位置  $\mathbf{X}_{e1}, \mathbf{X}_{e2}, \mathbf{X}_{e3}$ , 当前最优位置  $\mathbf{X}_{o1}, \mathbf{X}_{o2}, \mathbf{X}_{o3}$ , 连续陷入局部最优轮数  $n$ 、选择概率  $sp$  和初始能量  $E_0$ ;
6    For (each  $\mathbf{X}_i$ ) do {
7      用式(3)更新逃逸能量  $E$ ;
8      If  $E \geq 1$  then
9        进入探索阶段, 用式(25)更新位置;
10     Else if  $E < 1$  then {
11       If  $r < 0.1 + 0.8 \times sp$  then
12         进入开发阶段, 进行精英交叉演化, 用式(21)更新位置;
13       Else if  $r < 0.1 + 0.8 \times sp$  then
14         进入开发阶段, 进行精英融合演化, 用式(23)更新位置; }
15 Return  $\mathbf{X}_{e1}$ 

```

3.5 EHHO 时间复杂度分析

HHO 算法的时间复杂度主要由3个部分组成: 初始化、适应度值计算和哈里斯鹰位置更新。当种群数量为 N , 最大迭代次数为 T , 问题的维度为 D 时, 初始化的时间复杂度为 $O(N)$, 计算适应

度值并寻找历史最优位置的时间复杂度为 $O(N \times T)$, 哈里斯鹰位置更新的时间复杂度为 $O(N \times T \times D)$ 。因此, HHO 的时间复杂度为 $O(N \times T \times D)$ 。

EHHO 算法的时间复杂度主要由 5 个部分组成: 初始化、精英反向学习、适应度值计算、自适应机制运行和哈里斯鹰位置更新。当种群数量为 N , 最大迭代次数为 T , 问题的维度为 D 时, 初始化的时间复杂度为 $O(N)$, 精英反向学习的时间复杂度为 $O(N \times T)$, 计算适应度值并寻找当前最优位置和历史最优位置的时间复杂度为 $O(N \times T)$, 哈里斯鹰位置更新的时间复杂度为 $O(N \times T \times D)$, 自适应机制运行的时间复杂度为 $O(T)$ 。因此, EHHO 的时间复杂度为 $O(N \times T \times D)$ 。

综上所述, EHHO 与 HHO 的时间复杂度相同, 改进策略没有增加算法时间复杂度。

4 实验与结果分析

4.1 实验环境与基准函数

所有仿真实验都在相同环境下进行: 实验设备为一台安装 Windows 11 64 位家庭中文版操作系统的电脑, 运行内存为 16 GB, 处理器为 AMD Ryzen™ 7 5800H 3.2 GHz, 测试软件为 MATLAB R2020a。所有优化算法均采用相同的公共参数进行测试: 种群大小设置为 30, 最大迭代次数设

置为 500。为了减少实验中随机因素造成的偏差, 所有对比算法在每个测试函数上均独立运行 30 次, 用其寻优结果的平均值 (AVG) 和标准差 (STD) 来评估算法寻优性能和稳定性。

为了评估 EHHO 算法的探索能力、开发能力以及跳出局部最优的能力, 本文选取文献[16,17]中的 15 个基准函数, 这组基准函数包括单峰函数和多峰函数。其中, $F1 \sim F7$ 为单峰函数, 只有一个局部极值, 可以揭示不同优化算法的开发能力; $F8 \sim F15$ 为多峰函数, 具有多个局部极值, 可以揭示不同优化算法的探索能力和跳出局部最优的能力。基准函数的相关描述见表 1, 其中, 基准函数 $F1 \sim F10$ 为维数可变的函数, 选取维数 d 为 30 和 1 000 进行测试, 基准函数 $F11 \sim F15$ 为固定维函数。

4.2 改进策略有效性分析

为了充分验证本文改进策略的有效性, 将基本 HHO 算法与引入精英反向学习的 HHO1 算法、引入精英演化策略的 HHO2 算法、引入精英反向学习与精英演化策略的 HHO3 算法以及引入精英反向学习、精英演化策略与自适应机制的 EHHO 算法进行对比实验。实验结果如表 2 所示, 其中, 函数 $F1 \sim F7$ 为 30 维单峰函数, 函数 $F8 \sim F15$ 为 30 维和固定维多峰函数。

由表 2 可知, 对于函数 $F1$, 所有改进算法都能稳定收敛到理论最优值; 对于函数 $F2$ 和 $F4$, HHO1 算法的收敛精度提升了约 150 个数量级, HHO2 算法的收敛精度提升了约 180 个数量级,

Table 1 Benchmark functions

表 1 基准函数

函数	函数说明	维数	解空间	理论最优值
$F1$	Sphere Function	30, 1 000	$[-100, 100]$	0
$F2$	Schwefel's Problem 2.22	30, 1 000	$[-10, 10]$	0
$F3$	Schwefel's Problem 1.2	30, 1 000	$[-100, 100]$	0
$F4$	Schwefel's Problem 2.21	30, 1 000	$[-100, 100]$	0
$F5$	Generalized Rosenbrock's Function	30, 1 000	$[-30, 30]$	0
$F6$	Step Function	30, 1 000	$[-100, 100]$	0
$F7$	Quartic Function i. e. Noise	30, 1 000	$[-128, 128]$	0
$F8$	Generalized Schwefel's Problem 2.26	30, 1 000	$[-500, 500]$	$-418.9829 \times d$
$F9$	Generalized Penalized Function 1	30, 1 000	$[-50, 50]$	0
$F10$	Generalized Penalized Function 2	30, 1 000	$[-50, 50]$	0
$F11$	Shekel's Foxholes Function	2	$[-65, 65]$	0.998
$F12$	Kowalik's Function	4	$[-5, 5]$	0.000 30
$F13$	Branin Function	2	$[-5, 5]$	0.398
$F14$	Hartman's Family 2	6	$[0, 1]$	-3.32
$F15$	Shekel's Family 2	4	$[0, 10]$	-10.153 2

Table 2 Test results of different optimization strategies
表 2 不同优化策略的测试结果

函数	HHO		HHO1		HHO2		HHO3		EHHO	
	AVG	STD	AVG	STD	AVG	STD	AVG	STD	AVG	STD
F1	4.94E-94	2.70E-93	0	0	0	0	0	0	0	0
F2	6.52E-51	3.15E-50	7.06E-197	0	1.80E-239	0	0	0	0	0
F3	1.56E-74	7.81E-74	1.24E-165	0	0	0	0	0	0	0
F4	5.58E-48	2.62E-47	5.22E-207	0	8.26E-235	0	0	0	0	0
F5	1.36E-02	2.03E-02	3.33E-05	5.77E-05	1.69E-03	1.74E-03	1.32E-05	2.72E-05	3.53E-06	4.58E-06
F6	1.46E-04	2.05E-04	1.65E-07	3.28E-07	4.20E-07	2.74E-07	2.40E-08	2.27E-08	7.21E-10	8.36E-10
F7	1.58E-04	2.01E-04	6.43E-05	6.91E-05	1.76E-04	1.87E-04	5.39E-05	6.49E-05	3.81E-05	4.23E-05
F8	-12 541.42	1.50E+02	-12 569.48	4.47E-03	-12 569.47	1.13E-02	-12 569.48	1.87E-03	-12 569.49	5.17E-05
F9	8.25E-06	7.38E-06	1.85E-08	5.08E-08	4.99E-08	2.98E-08	4.24E-09	3.72E-09	1.17E-10	1.70E-10
F10	1.06E-04	1.39E-04	1.18E-07	2.33E-07	8.07E-07	5.24E-07	4.48E-08	4.39E-08	2.60E-09	3.74E-09
F11	1.361 17	9.52E-01	0.998 00	7.16E-11	0.998 00	3.29E-16	0.998 00	4.03E-16	0.998 00	3.13E-16
F12	0.000 37	1.74E-04	0.000 33	1.51E-05	0.000 34	1.69E-04	0.000 31	2.30E-06	0.000 31	1.00E-06
F13	0.397 89	2.56E-05	0.397 89	1.08E-06	0.397 89	1.35E-15	0.397 89	1.11E-15	0.397 89	0
F14	-3.121 1	1.06E-01	-3.086 7	1.05E-01	-3.274 4	5.92E-02	-3.322 0	4.67E-12	-3.322 0	1.89E-15
F15	-5.367 0	1.20E+00	-10.153 2	4.82E-05	-10.153 2	3.11E-11	-10.153 2	1.53E-11	-10.153 2	4.38E-15

而 HHO3 算法与 EHHO 算法能稳定收敛到理论最优值;对于函数 F3,HHO1 算法的寻优精度提升了约 90 个数量级,而其余改进算法都能稳定寻找到理论最优值;对于函数 F5、F6、F8、F9、F10,所有改进算法的收敛精度都提升了 1~6 个数量级;对于函数 F7 和 F12,HHO1 算法、HHO3 算法和 EHHO 算法的收敛精度都提升了 1~2 个数量级;对于函数 F11,HHO1 算法的收敛精度提升了 10 个数量级,HHO2 算法、HHO3 算法和 EHHO 算法的收敛精度都提升了 15 个数量级;对于函数 F13,HHO2 算法和 HHO3 算法的收敛精度提升了 10 个数量级,EHHO 算法能稳定收敛到理论最优值;对于函数 F14,HHO3 算法的收敛精度提升了 11 个数量级,EHHO 算法的收敛精度提升了 14 个数量级;对于函数 F15,HHO1 算法的收敛精度提升了 5 个数量级,HHO2 算法和 HHO3 算法的收敛精度提升了 11 个数量级,EHHO 算法的收敛精度提升了 15 个数量级。

综上,引入精英反向学习和引入精英演化策略都能在一定程度上增强算法探索能力、开发能力和跳出局部最优的能力,进而提升算法收敛精度;同时引入精英反向学习和精英演化策略能够融合 2 种改进策略的优势,对算法性能的提升比单独使用一种策略时更大;而在引入精英反向学习和精英演化策略的同时引入自适应机制能够更进一步提升算法寻优性能和稳定性,即 3 种改进策略对算法性

能的提升均有一定效果。

4.3 性能对比分析

为了充分验证本文改进算法的有效性,将 EHHO 算法与其他最新的优化算法进行对比实验,对比算法包括平衡优化算法 EO(Equilibrium Optimizer)^[18]、模因哈里斯鹰优化 EESHHO (Elite Evolution Strategy with Harris Hawks Optimization)算法、改进的灰狼优化算法 IGWO(Improved Grey Wolf Optimizer)^[19]和人工大猩猩部队优化算法 GTO(Gorilla Troops Optimizer)^[20]。实验结果如表 3 所示,其中,函数 F1~F7 为 30 维单峰函数,函数 F8~F15 为 30 维和固定维多峰函数。

由表 3 可知,EHHO 算法在 30 维单峰函数上寻优性能优异。对于函数 F1,EESHHO 算法、GTO 算法和 EHHO 算法都能稳定收敛到理论最优值;对于函数 F2 和 F4,仅 EHHO 算法能稳定收敛到理论最优值;对于函数 F3,仅 EHHO 算法和 GTO 算法能收敛到理论最优值;对于函数 F5、F6 和 F7,EHHO 算法的收敛精度和稳定性均高于对比算法。

由表 3 可知,EHHO 算法在 30 维和固定维多峰函数上寻优性能优异。对于函数 F8、F9、F10 和 F12,EHHO 算法的收敛精度和稳定性都高于对比算法;对于函数 F11,EHHO 算法的收敛精度与其他对比算法持平,稳定性比 GTO 算法略低;

Table 3 Test results of different optimization algorithms
表 3 不同优化算法的测试结果

函数	EO		EESHHO		IGWO		GTO		EHHO	
	AVG	STD	AVG	STD	AVG	STD	AVG	STD	AVG	STD
F1	5.66E-40	2.66E-39	0	0	1.70E-28	1.95E-28	0	0	0	0
F2	8.94E-23	1.14E-22	1.81E-190	0	6.14E-18	5.97E-18	2.79E-192	0	0	0
F3	1.54E-09	7.37E-09	5.41E-225	0	8.84E-04	1.42E-03	0	0	0	0
F4	5.45E-10	1.45E-09	1.57E-156	7.57E-156	1.60E-05	1.42E-05	2.67E-196	0	0	0
F5	2.54E+01	2.34E-01	8.89E-01	4.69E+00	2.45E+01	8.93E-01	3.21E+00	8.27E+00	3.53E-06	4.58E-06
F6	8.77E-06	5.80E-06	1.11E-05	7.46E-06	2.78E-02	7.45E-02	8.79E-08	1.00E-07	7.21E-10	8.36E-10
F7	1.24E-03	6.32E-04	1.80E-04	1.34E-04	2.86E-03	1.25E-03	1.01E-04	6.92E-05	3.81E-05	4.23E-05
F8	-8 795.39	5.94E+02	-11 640.97	1.06E+03	-8 054.05	1.55E+03	-12 569.49	1.57E-04	-12 569.49	5.17E-05
F9	5.82E-07	5.77E-07	6.20E-07	4.66E-07	3.77E-03	1.89E-02	3.06E-08	4.82E-08	1.17E-10	1.70E-10
F10	3.43E-02	6.06E-02	5.42E-06	3.72E-06	1.26E-01	1.33E-01	2.17E-03	5.20E-03	2.60E-09	3.74E-09
F11	0.998 00	1.37E-16	0.998 00	3.07E-16	0.998 00	2.06E-16	0.998 00	4.12E-17	0.998 00	3.02E-16
F12	0.003 78	7.55E-03	0.000 40	2.79E-04	0.000 33	1.27E-04	0.000 37	2.32E-04	0.000 31	1.77E-06
F13	0.397 89	0	0.397 89	8.95E-14	0.397 89	0	0.397 89	0	0.397 89	0
F14	-3.264 4	7.08E-02	-3.282 36	5.70E-02	-3.322 00	5.67E-07	-3.290 29	5.35E-02	-3.322 00	1.23E-15
F15	-8.793 7	2.29E+00	-10.153 2	8.36E-14	-9.811 1	1.11E+00	-10.153 2	5.83E-15	-10.153 2	4.28E-15

对于函数 F13,EHHO 算法、EO 算法、IGWO 算法和 GTO 算法能稳定收敛到理论最优值;对于函数 F14,EHHO 算法和 IGWO 算法收敛精度最高,且 EHHO 算法稳定性高于 IGWO 算法;对于函数 F15,EHHO 算法、GTO 算法和 EESHHO 算法收敛精度和稳定性明显高于其他对比算法。

综上,总体上 EHHO 算法在低维情形下寻优性能良好,收敛精度均不劣于对比算法,稳定性高。

图 4a~图 4d 是对比算法在部分 30 维单峰函数上独立运行 30 次的平均收敛曲线。对于函数 F1,EHHO 算法在约 220 次迭代时收敛精度达到最高,收敛速度明显快于对比算法;对于函数 F4,

EHHO 算法在约 400 次迭代时收敛精度达到最高,收敛速度明显快于对比算法,收敛精度比对比算法高 100~300 个数量级;对于函数 F5 和 F6,EHHO 算法在迭代过程中未陷入局部最优,收敛速度快于对比算法,收敛精度比对比算法高 3 个以上数量级。

图 4e~图 4h 是对比算法在部分 30 维多峰和固定维多峰函数上独立运行 30 次的平均收敛曲线。对于函数 F8、F14 和 F15,EHHO 算法收敛精度高于或等于对比算法,且收敛速度快于对比算法;对于函数 F10,EHHO 算法在迭代过程中未陷入局部最优,收敛速度快于对比算法,收敛精度比

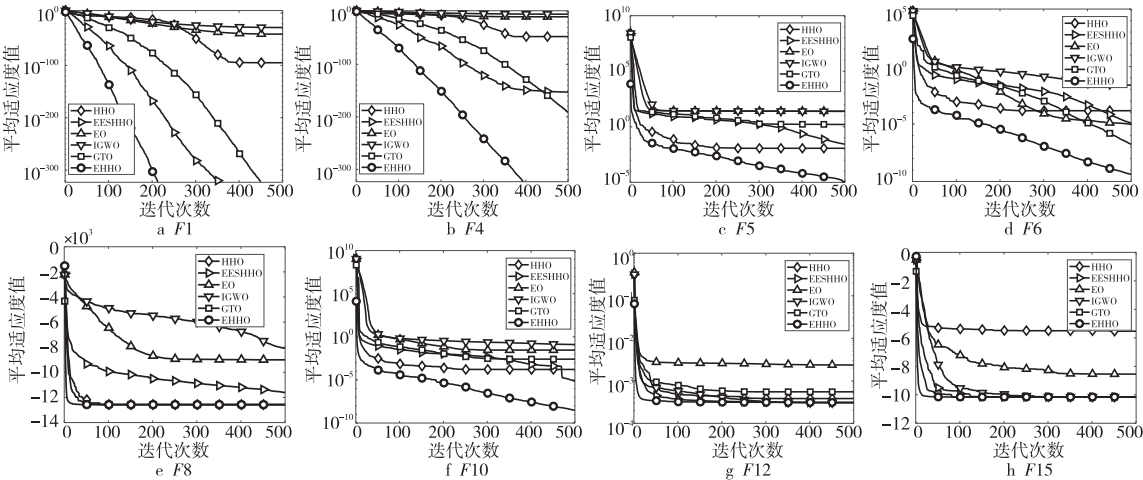


Figure 4 Convergence curves of benchmark functions
图 4 基准函数收敛曲线

对比算法高 2 个以上数量级。

综上,总体上 EHHO 算法在低维情形下寻优过程顺利,收敛速度快,跳出局部最优的能力强。

4.4 Wilcoxon 秩和检验

为了进一步验证本文改进算法的有效性,在显著性水平 $P=5\%$ 的条件下,本节用 Wilcoxon 秩和检验来比较 EHHO 算法与其他群智能算法在 15 个基准函数上的运行结果之间是否存在显著性差异。符号“+”表示 EHHO 算法优于对比算法;符号“=”表示 EHHO 算法相当于对比算法;符号“-”表示 EHHO 算法劣于对比算法;“NaN”表示不适用,无法判定比较数据是否存在显著性差异。

实验结果如表 4 所示,EHHO 算法与 HHO 算法、EO 算法和 IGWO 算法在 14 个基准函数上的运行结果存在显著性差异;EHHO 算法与 EESHHO 算法在 12 个测试函数上的运行结果存在显著性差异;EHHO 算法与 GTO 算法在 10 个测试函数上的运行结果存在显著性差异。

综上,总体上 EHHO 算法与对比算法存在显著性差异,EHHO 算法寻优性能优于对比算法的。

4.5 高维情形下性能分析

为了进一步验证改进算法的有效性,本节在高维情形下将 EHHO 算法与 HHO 算法、EO 算法、EESHHO 算法、IGWO 算法和 GTO 算法进行对比

实验,实验结果如表 5 所示。

由表 5 可知,EHHO 算法在 1 000 维单峰和多峰测试函数的测试中,寻优性能与对比算法持平或优于对比算法。对于函数 $F1$ 和 $F3$,仅 EHHO 算法和 GTO 算法能稳定收敛到理论最优值;对于函数 $F2$,仅 EHHO 算法和 EESHHO 算法能稳定收敛到理论最优值;对于函数 $F4$,仅 EHHO 算法能稳定收敛到理论最优值;对于函数 $F5\sim F10$,EHHO 算法在收敛精度和稳定性上都优于对比算法。

综上,EHHO 算法在高维情形下仍能保持良好的寻优性能,收敛精度和稳定性均不劣于对比算法。

5 结束语

针对哈里斯鹰优化算法易陷入局部最优和收敛精度低的问题,本文提出一种基于精英引导的改进哈里斯鹰优化算法。引入精英反向学习,以精英中心为对称中心生成反向解,并优先保留反向解以优化种群结构,增强算法跳出局部最优的能力;引入精英演化策略,用精英交叉演化与精英融合演化来更新种群位置,以提升种群质量,增强算法局部开发能力,提升算法收敛精度;引入自适应机制,动态调节选择 2 种精英演化方法更新种群位置的概

Table 4 Tests results of Wilcoxon rank sum test

表 4 Wilcoxon 秩和检验结果

函数	HHO			EESHHO			EO			IGWO			GTO		
	P-value	+/-/=		P-value	+/-/=		P-value	+/-/=		P-value	+/-/=		P-value	+/-/=	
F1	1.21E-12	+		NaN	=		1.21E-12	+		1.21E-12	+		NaN	=	
F2	1.21E-12	+		1.21E-12	+		1.21E-12	+		1.21E-12	+		1.21E-12	+	
F3	1.21E-12	+		1.21E-12	+		1.21E-12	+		1.21E-12	+		NaN	=	
F4	1.21E-12	+		1.21E-12	+		1.21E-12	+		1.21E-12	+		1.21E-12	+	
F5	4.08E-11	+		3.02E-11	+		3.02E-11	+		3.02E-11	+		5.97E-05	+	
F6	3.02E-11	+		3.02E-11	+		3.02E-11	+		3.02E-11	+		3.02E-11	+	
F7	1.37E-01	=		1.63E-02	+		3.02E-11	+		3.02E-11	+		8.07E-01	=	
F8	5.57E-10	+		3.02E-11	+		3.02E-11	+		3.02E-11	+		2.23E-01	=	
F9	3.02E-11	+		3.02E-11	+		3.02E-11	+		3.02E-11	+		1.33E-10	+	
F10	3.02E-11	+		3.02E-11	+		3.02E-11	+		3.02E-11	+		3.82E-09	+	
F11	1.45E-11	+		5.12E-02	=		1.30E-05	+		1.45E-11	+		3.74E-09	-	
F12	5.49E-11	+		7.48E-02	=		3.00E-07	+		2.61E-10	+		1.05E-07	+	
F13	4.57E-12	+		2.79E-03	+		NaN	=		1.21E-12	+		NaN	=	
F14	2.05E-11	+		2.62E-11	+		3.37E-02	+		2.05E-11	+		1.79E-02	+	
F15	2.02E-11	+		8.99E-09	+		2.04E-02	+		2.02E-11	+		1.43E-07	+	

Table 5 Test results in high-dimensional cases(1 000D)
表 5 高维情形下的测试结果(1 000D)

函数	指标	HHO	EO	EESHHO	IGWO	GTO	EHHO
F1	AVG	1.46E-96	6.21E-21	2.11e-317	2.96E+00	0.00E+00	0.00E+00
	STD	4.72E-96	5.59E-21	0.00E+00	1.04E+00	0.00E+00	0.00E+00
F2	AVG	9.04E-47	5.07E-13	0.00E+00	Inf	3.37E-189	0.00E+00
	STD	4.91E-46	3.62E-13	0.00E+00	NaN	0.00E+00	0.00E+00
F3	AVG	1.01E-27	2.64E+05	2.62E-175	2.91E+06	0.00E+00	0.00E+00
	STD	5.53E-27	3.75E+05	0.00E+00	3.31E+05	0.00E+00	0.00E+00
F4	AVG	1.59E-49	8.04E+01	2.97E-132	6.60E+01	5.96E-184	0.00E+00
	STD	4.42E-49	9.70E+00	1.63E-131	2.40E+00	0.00E+00	0.00E+00
F5	AVG	2.60E-01	9.97E+02	2.74E+02	2.32E+07	3.41E+00	1.55E-03
	STD	3.65E-01	3.95E-01	2.51E+02	8.74E+06	6.27E+00	3.39E-03
F6	AVG	3.80E-03	2.07E+02	1.79E+00	2.04E+02	1.05E+00	1.34E-05
	STD	5.34E-03	2.29E+00	1.54E+00	4.33E+00	1.23E+00	5.29E-05
F7	AVG	2.70E-04	4.99E-03	2.54E-04	1.95E+02	1.05E-04	7.02E-05
	STD	2.59E-04	1.94E-03	2.35E-04	9.03E+01	7.14E-05	5.45E-05
F8	AVG	-4.19E+05	-1.15E+05	-3.64E+05	-5.31E+04	-4.19E+05	-4.19E+05
	STD	4.44E+01	8.78E+03	5.12E+04	3.67E+04	6.64E+01	2.21E+00
F9	AVG	2.55E-06	8.20E-01	6.41E-04	1.45E+07	2.42E-04	3.24E-09
	STD	3.62E-06	2.23E-02	5.54E-04	6.92E+06	2.59E-04	5.59E-09
F10	AVG	1.12E-03	9.93E+01	3.04E-01	7.04E+07	1.35E-01	3.62E-06
	STD	1.21E-03	2.35E-01	2.28E-01	3.48E+07	2.89E-01	6.28E-06

率,以增强算法稳定性。选取 15 个基准函数进行仿真实验,实验结果表明,EHHO 算法采取的改进策略都能有效提升算法寻优性能。虽然本文改进策略一定程度上增加了算法的复杂度,但改进后算法收敛速度快、寻优精度高、鲁棒性强,在优化算法中具有明显优势。接下来的研究将考虑如何进一步优化改进算法,将其与实际工程应用相结合。

参考文献:

[1] Abbassi R,Abbassi A,Heidari A A, et al. An efficient salp swarm-inspired algorithm for parameters identification of photovoltaic cell models [J]. Energy Conversion and Management,2019,179(3):62-72.

[2] Faris H, AI-Zoubi A M, Heidari A A, et al. An intelligent system for spam detection and identification of the most relevant features based on evolutionary random weight networks [J]. Information Fusion,2019,48(C):67-83.

[3] Kennedy J, Eberhart R. Particle swarm optimization[C]// Proc of International Conference on Neural Networks,1995: 1942-1948.

[4] 钟倩漪,钱谦,伏云发,等. 粒子群优化算法在关联规则挖掘中的研究综述[J]. 计算机科学与探索,2021,15(5):777-793. Zhong Qian-yi, Qian Qian, Fu Yun-fa, et al. Survey of particle swarm optimization algorithm for association rule mining[J].

Journal of Frontiers of Computer Science and Technology, 2021,15(5): 777-793.

[5] Mirjalili S,Lewis A. The whale optimization algorithm [J]. Advances in Engineering Software,2016,9(5):51-67.

[6] 李丹. 改进群智能优化算法的海上物流配送路径优化方法 [J]. 舰船科学技术,2020,42(8A):184-186. Li Dan. Optimization method of marine logistics distribution path based on improved swarm intelligence optimization algorithm[J]. Ship Science and Technology, 2020, 42(8A): 184-186.

[7] Wolpert D H,Macready W G. No free lunch theorems for optimization [J]. IEEE Transactions on Evolutionary Computation,1997,1(1): 67-82.

[8] 张九龙,王晓峰,芦磊,等. 若干新型智能优化算法对比分析研究[J]. 计算机科学与探索,2022,16(1):88-105. Zhang Jiu-long, Wang Xiao-feng, Lu Lei, et al. Analysis and research of several new intelligent optimization algorithms [J]. Journal of Frontiers of Computer Science and Technology,2022,16(1): 88-105.

[9] Heidari A A,Mirjalili S,Faris H,et al. Harris hawks optimization: Algorithm and applications [J]. Future Generation Computer Systems,2019,97(8):49-72.

[10] Li C,Li J,Chen H, et al. Memetic Harris hawks optimization: Developments and perspectives on project scheduling and QoS-aware web service composition [J]. Expert Systems with Applications,2021,171:114529.

- [11] 郭雨鑫,刘升,高文欣,等.精英反向学习与黄金正弦优化的HHO算法[J].计算机工程与应用,2022,58(10):153-161.
Guo Yu-xin, Liu Sheng, Gao Wen-xin, et al. Elite opposition-based learning golden-sine Harris hawks optimization[J]. Computer Engineering and Applications, 2022, 58(10): 153-161.
- [12] 陈功,曾国辉,黄勃,等.融合互利共生和透镜成像学习的HHO算法[J].计算机工程与应用,2022,58(10):76-86.
Chen Gong, Zeng Guo-hui, Huang Bo, et al. Harris hawks optimization algorithm combining mutualism and lens imaging learning[J]. Computer Engineering and Applications, 2022, 58(10): 76-86.
- [13] 汤安迪,韩统,徐登武,等.混沌精英哈里斯鹰优化算法[J].计算机应用,2021,41(8):2265-2272.
Tang An-di, Han Tong, Xu Deng-wu, et al. Chaotic elite Harris hawks optimization algorithm[J]. Journal of Computer Applications, 2021, 41(8): 2265-2272.
- [14] 朱诚,潘旭华,张勇.基于趋化校正的哈里斯鹰优化算法[J].计算机应用,2022,42(4):1186-1193.
Zhu Cheng, Pan Xu-hua, Zhang Yong. Harris hawks optimization algorithm based on chemotaxis correction[J]. Journal of Computer Applications, 2022, 42(4): 1186-1193.
- [15] Tizhoosh H R. Opposition-based learning: A new scheme for machine intelligence[C]//Proc of International Conference on Computational Intelligence for Modelling, Control and Automation and International Conference on Intelligent Agents, Web Technologies and Internet Commerce, 2005: 695-701.
- [16] Digalakis J G, Margaritis K G. On benchmarking functions for genetic algorithms [J]. International Journal of Computer Mathematics, 2001, 77(4): 481-506.
- [17] Yao X, Liu Y, Lin G. Evolutionary programming made faster [J]. IEEE Transactions on Evolutionary Computation, 1999, 3(2): 82-102.
- [18] Faramarzi A, Heidarinejad M, Stephens B, et al. Equilibrium optimizer: A novel optimization algorithm [J]. Knowledge-Based Systems, 2020, 191: 105190.
- [19] Nadimi-shahraki M H, Taghiani S, Mirjalili S. An improved grey wolf optimizer for solving engineering problems [J]. Expert Systems with Applications, 2021, 166: 113917.
- [20] Abdollahzadeh B, Gharehchopogh F S, Mirjalili S. Artificial gorilla troops optimizer: A new nature-inspired metaheuristic algorithm for global optimization problems [J]. International Journal of Intelligent Systems, 2021, 36(10): 5887-5958.

作者简介:



李雨恒(1997-),男,四川大竹人,硕士生,CCF会员(K5938G),研究方向为智能计算。**E-mail:**jkloplyh@qq.com

LI Yu-heng, born in 1997, MS candidate, CCF member (K5938G), his research interest includes intelligent computing.



高尚(1972-),男,安徽天长人,博士,教授,CCF会员(30644M),研究方向为智能计算、模式识别和人工智能。**E-mail:**gaoshang@sohu.com

GAO Shang, born in 1972, PhD, professor, CCF member (30644M), his research interests include intelligent computing, pattern recognition, and artificial intelligence.



孟祥宇(1996-),男,江苏赣榆人,硕士生,研究方向为机器学习。**E-mail:**1426987298@qq.com

MENG Xiang-yu, born in 1996, MS candidate, his research interest includes machine learning.